

Garage Abracadabra

Internet of Things, 2024/2025

Realizzato dal **Gruppo 16:**

Pagano Maria Pia

Senatore Cory

Volzone Valerio

FUNZIONALITÀ

Il sistema realizza un prototipo di parcheggio pubblico *smart* dotato di un singolo posto auto.



- L'avvicinamento di un'auto dall'esterno fa sollevare la sbarra, se il posto è disponibile, e attiva la riproduzione dell'omonima canzone.
- L'ingresso del veicolo viene rilevato e, al termine della canzone, la sbarra si richiude se il veicolo si è spostato. Lo schermo esterno segnala che il posto è occupato.
 - In questa condizione, se un'altra auto si avvicina, la sbarra non si alza.
- Per consentire l'uscita, è necessario effettuare il pagamento, che può avvenire sia fisicamente sia da remoto.
- Un sistema di allarme consente l'apertura della sbarra in condizioni meteo estreme, con soglie configurabili tramite dashboard.
- È inoltre possibile effettuare da remoto la prenotazione del posto, che impedisce l'accesso ad altri veicoli. La prenotazione può essere interrotta all'occorrenza.

COMPONENTI

Il funzionamento del prototipo è reso possibile grazie al microcontrollore **ESP32**, che gestisce un insieme di sensori in grado di rilevare dati dalla realtà fisica.

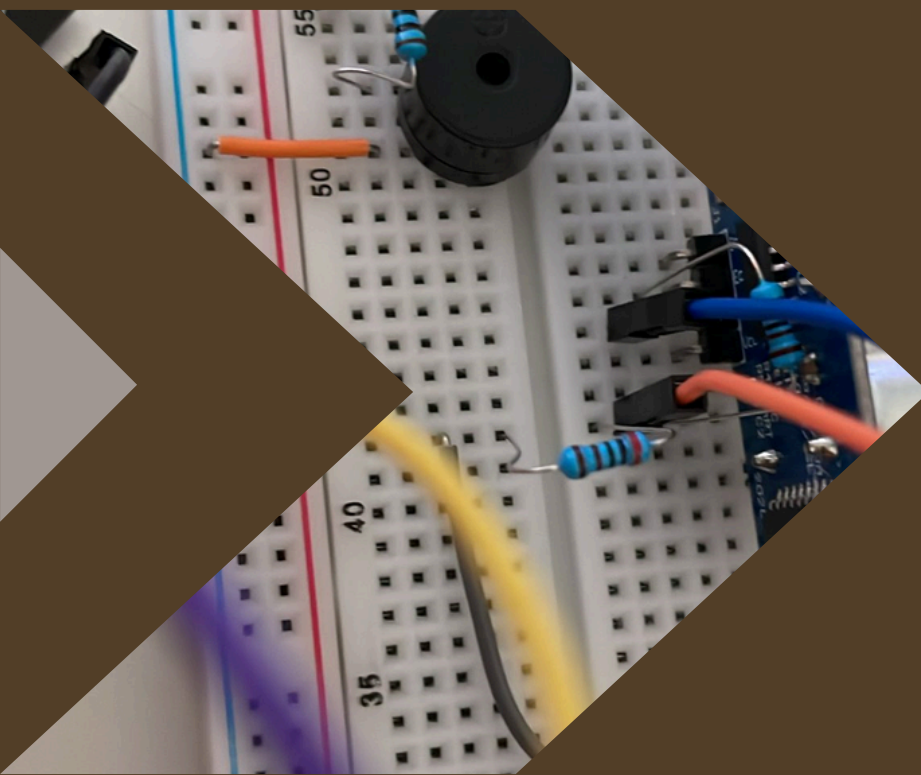
Queste informazioni vengono poi utilizzate dalla logica applicativa, implementata in **MicroPython**.

Per la comunicazione da remoto è stato adottato il **protocollo MQTT**, che, tramite connessione Wi-Fi, consente al dispositivo di interagire con altri nodi presenti sulla rete.



COMPONENTI

- 2 sensori a ultrasuoni HC-SR04: rilevano la presenza dell'auto all'esterno e all'interno del parcheggio.
- Display OLED: mostra il logo e lo stato attuale del parcheggio (libero, occupato o prenotato).
- Servomotore: controlla l'apertura e la chiusura della sbarra.
- Semaforo a LED: indica in modo intuitivo lo stato del parcheggio tramite i colori rosso, giallo e verde.
- Buzzer passivo: riproduce la canzone "Abracadabra" di Lady Gaga durante l'apertura della sbarra e fornisce un segnale acustico in caso di allarme.
- Sensore DHT22: misura temperatura e umidità, attivando l'allarme in presenza di condizioni meteo estreme.
- Bottone: simula il pagamento locale del parcheggio.
- Broker MQTT in esecuzione su Docker locale: gestisce la comunicazione tra i dispositivi tramite pubblicazione e sottoscrizione ai topic del protocollo MQTT.
- Dashboard Node-RED: consente di gestire prenotazioni, pagamenti e le soglie ambientali del sistema di allarme.



SENSORE HCSR04

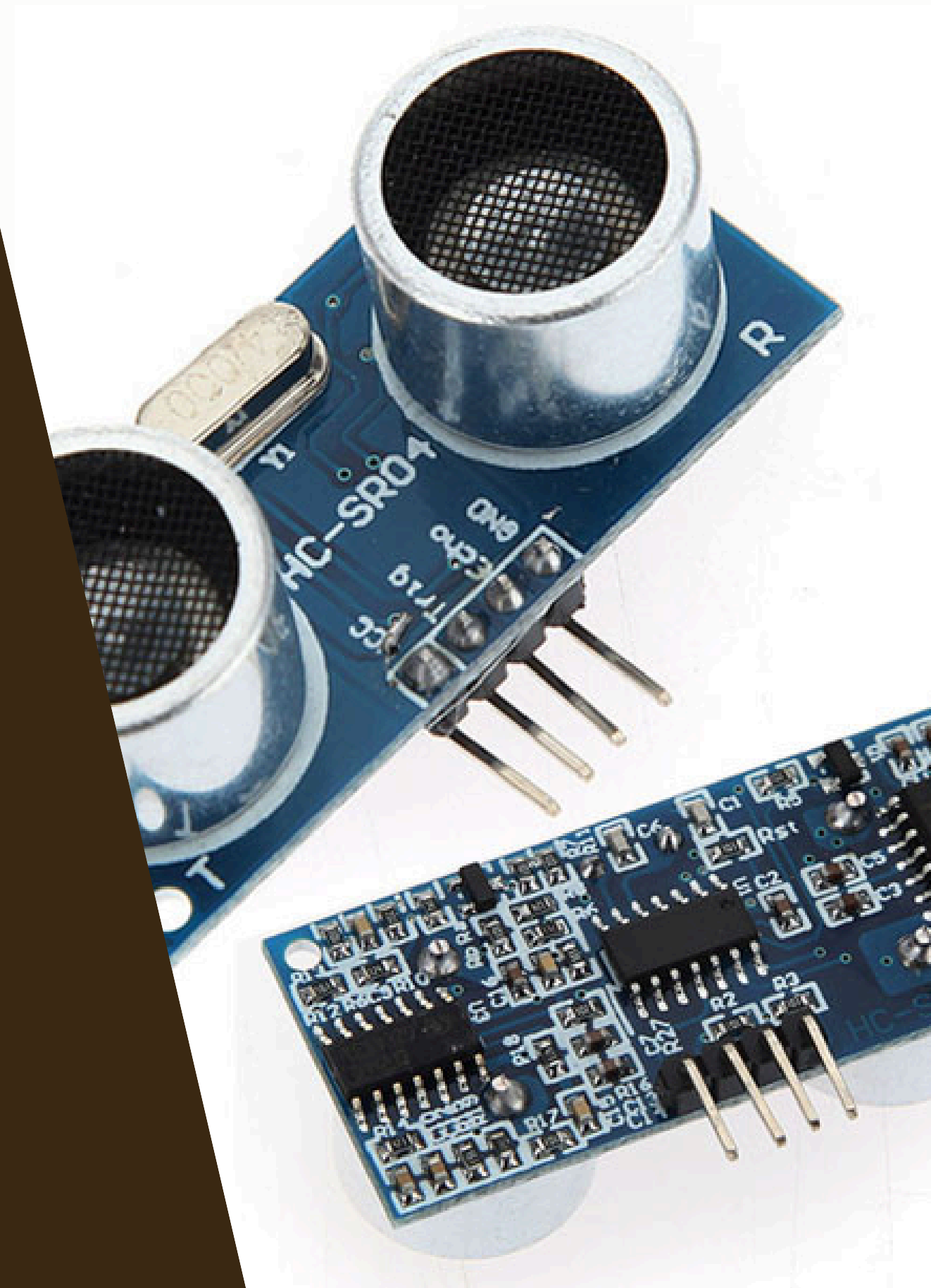
I due sensori a ultrasuoni **HC-SR04** sono utilizzati per rilevare la presenza di un veicolo sia in fase di avvicinamento alla sbarra che all'interno dell'area di sosta.

Ogni sensore è collegato a un **partitore di tensione** composto da una resistenza da 1 k Ω e una da 2 k Ω , utilizzato per adeguare il livello di uscita del pin ECHO, che fornisce un segnale a 5V, alla logica compatibile con l'ESP32, che opera a 3.3V.

In particolare, il partitore consente di abbassare la tensione del segnale **ECHO** a un livello sicuro per il microcontrollore.

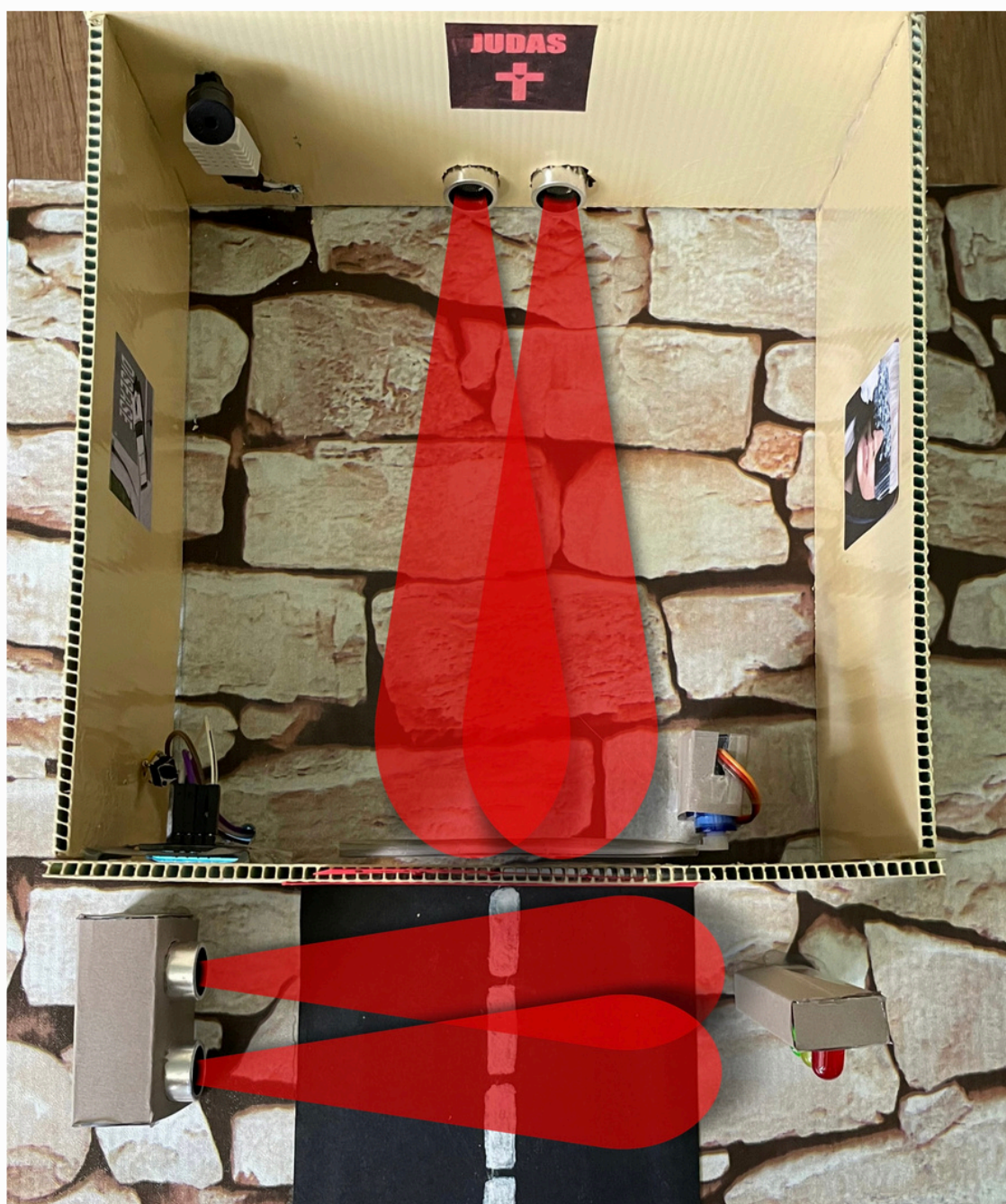
I pin **TRIGGER** ed **ECHO** di ciascun sensore sono collegati a pin GPIO dell'ESP32, che tramite *MicroPython* gestisce la lettura dei dati di distanza.

Il pin GND di ciascun sensore è connesso alla massa comune sulla breadboard, mentre il pin VCC è alimentato a 5V.



FUNZIONAMENTO dei sensori HC SR04

LA LOGICA DEL SISTEMA PREVEDE TRE TIPOLOGIE DI VERIFICA:



1. Verifica parcheggio libero: il sensore interno rileva l'assenza di un veicolo nel posto auto, indicando che lo spazio è disponibile.
2. Verifica parcheggio occupato: il sensore interno rileva la presenza di un veicolo nel posto auto, aggiornando lo stato a "occupato".
3. Verifica avvicinamento veicolo: il sensore esterno monitora lo spazio antistante la sbarra, rilevando un veicolo in avvicinamento.

In presenza di un veicolo in avvicinamento (rilevato dal sensore esterno) e di una condizione di posto libero (verificata dal sensore interno), il sistema autorizza l'apertura automatica della barriera, se non è attiva una prenotazione.

In presenza di un veicolo all'interno del parcheggio, il sistema autorizza l'apertura della barriera solo dopo il pagamento (tramite dashboard oppure bottone).

SEMAFORO LED

A supporto del sistema di comunicazione basato sul display OLED, è stato integrato un **semaforo a LED** per fornire un **feedback visivo intuitivo** sullo stato del parcheggio.

Il modulo LED è alimentato con una tensione di 3.3V ed è collegato all'ESP32 tramite tre pin digitali dedicati, ciascuno associato a un canale:

- R (rosso)
- Y (giallo)
- G (verde)



SEMAFORO LED



LE DIVERSE CONDIZIONI DEL PARCHEGGIO SONO RAPPRESENTATE DALLE SEGUENTI LUCI:

- Luce verde: quando il sensore interno rileva che il posto auto è libero.
- Luce rossa: quando il posto è occupato da un veicolo o è prenotato.
- Luce gialla: quando il sistema è in attesa che il veicolo entri nel garage o esca da questo.
- Luci verde, gialla e rossa ad intermittenza: quando il sistema è in stato di allarme per condizioni meteo.

SERVO MOTOR

La gestione dell'apertura e chiusura della barriera del parcheggio è affidata a un servomotore.

Il servo presenta tre connessioni:

- **PIN di segnale**: connesso a un GPIO dell'ESP32, consente di regolare l'angolo di rotazione del motore tramite segnali PWM.
- **PIN GND**: collegato alla breadboard e messo a massa per completare il circuito.
- **PIN VCC**: alimentato a 3.3V, compatibile con l'output dell'ESP32.



SERVO MOTOR



Il *servo motor* opera secondo una logica di stato conseguente alle letture dei sensori a ultrasuoni e all'eventuale prenotazione via dashboard.

Il comportamento è :

- **Angolo di 0° , sbarra orizzontale**: posizione di chiusura, corrispondente allo stato di parcheggio libero, occupato o prenotato.
- **Angolo di 90° , sbarra verticale**: posizione di apertura, presente nel momento in cui una vettura si avvicina al parcheggio e questo risulta disponibile, successivamente al pagamento oppure quando il sistema è in stato di allarme per condizioni meteo.

Una volta raggiunto l'angolo di apertura (90°), il sistema riproduce la canzone "Abracadabra" per consentire il passaggio del veicolo, dopodiché vengono controllate nuovamente le misurazioni dei sensori ad ultrasuoni.

BUZZER PASSIVO



Nel sistema è stato integrato un **buzzer passivo** con lo scopo di fornire un **feedback sonoro** all'apertura della barriera ed al sistema di allarme.

Il buzzer è montato sulla breadboard ed è collegato in serie a una resistenza da 20Ω , per limitare la corrente e proteggere il componente.

Le connessioni elettriche del buzzer:

- **PIN di segnale**: collegato a un GPIO dell'ESP32, utilizzato per programmare le frequenze delle note.
- **PIN GND**: collegato alla linea di massa comune della breadboard.

BUZZER PASSIVO

La programmazione del buzzer è strettamente legata alla logica di apertura della sbarra. Quando i sensori a ultrasuoni rilevano che il posto auto è libero e viene individuato un veicolo in avvicinamento, oppure in seguito al pagamento, il sistema attiva il buzzer.

Il suono riprodotto è una sequenza di note programmata nel codice *MicroPython*, ispirata al brano "***Abracadabra***" di *Lady Gaga*, generata tramite l'emissione di frequenze specifiche in successione temporale.

In caso di condizioni meteo critiche rilevate all'interno del parcheggio, il buzzer viene invece utilizzato per emettere un **segnale di allarme**.



OLED SSD1306

Il sistema include un display OLED utilizzato per la visualizzazione di contenuti informativi in tempo reale. In condizioni di funzionamento ordinario, il display mostra il logo e lo stato del parcheggio (occupato/prenotato). In stato di allerta, invece, viene generato un messaggio di emergenza (ALERT!).

Il modulo OLED opera tramite protocollo I2C ed è collegato all'ESP32 secondo la seguente configurazione:

- **PIN VCC**: collegato alla linea di alimentazione a 3.3V
- **PIN GND**: collegato alla massa comune del circuito
- **PIN SDA**: collegato al pin GPIO19
- **PIN SCL**: collegato al pin GPIO18

Il display è programmato per aggiornarsi dinamicamente in base ai segnali ricevuti dai sensori e dallo stato del sistema, integrando le logiche di notifica visiva durante il normale funzionamento del sistema (temperatura/umidità).



DHT22

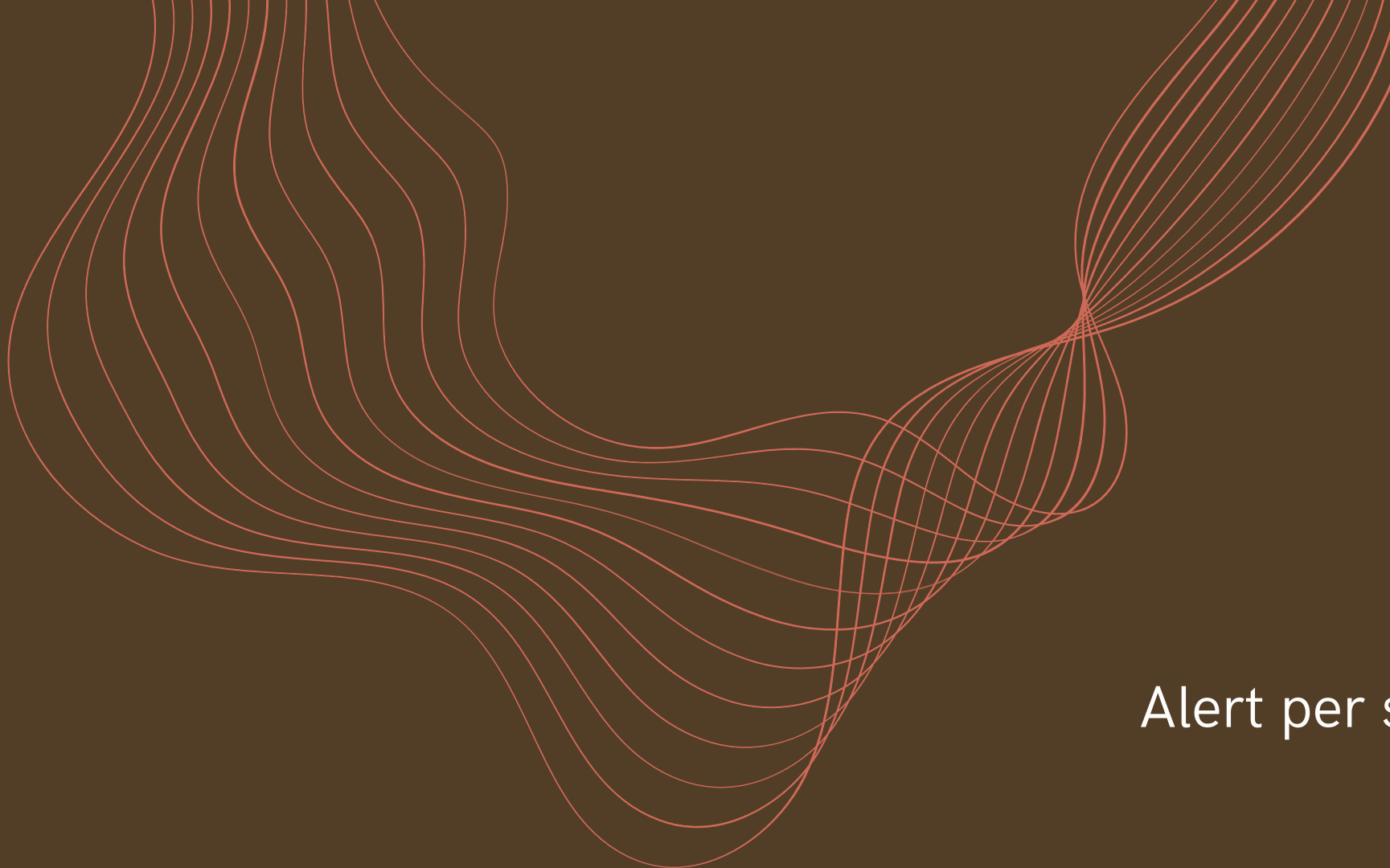
All'interno del sistema è stato integrato un sensore di umidità e temperatura, utile per monitorare le condizioni ambientali del parcheggio e prevenire eventuali situazioni di rischio legate al surriscaldamento o all'umidità eccessiva.

Collegamenti elettrici del DHT22:

- **PIN VCC**: collegato all'alimentazione a 3.3V.
- **PIN GND**: collegato alla massa comune della breadboard.
- **PIN DATA**: collegato a un GPIO dell'ESP32, con l'aggiunta di una resistenza di pull-up da 10kΩ tra VCC e DATA per la corretta trasmissione dei segnali.
- **PIN NC**: pin non connesso

Il microcontrollore effettua campionamenti periodici dei dati provenienti dal DHT22. I valori acquisiti vengono visualizzati in tempo reale su un display OLED e su *Dashboard Node-RED*. L'output mostra la **temperatura** in gradi **Celsius (°C)** e **l'umidità relativa** in **percentuale (%)**, aggiornati a intervalli regolari.





DHT22

Alert per sistema di allarme:

È stata implementata una logica condizionale che monitora i valori acquisiti dal sensore.

Quando Umidità $\geq$ soglia di umidità o Temperatura $\geq$ soglia di temperatura
(entrambe le soglie possono essere impostate tramite dashboard)

il sistema interpreta lo scenario come condizione ambientale critica e attiva un sistema di allarme.



SISTEMA DI ALLARME

Il sistema di allarme si attiva automaticamente al superamento di soglie ambientali critiche rilevate dal sensore DHT22.

Queste soglie, inizialmente impostate con valori predefiniti ma modificabili tramite la dashboard di *Node-RED*, consentono di identificare condizioni meteo potenzialmente pericolose.

COMPORTAMENTI DEL SISTEMA IN STATO DI ALLERTA:

01

Attivazione del buzzer d'allarme:

Il buzzer piezoelettrico emette un segnale acustico intermittente ad alta frequenza

02

Segnalazione visiva tramite LED:

I tre LED (rosso, giallo, verde) del semaforo vengono attivati in lampeggiamento

03

Apertura automatica della barriera:

Il servo motore viene attivato portando l'angolo a 90° per aprire la barriera

04

Visualizzazione di un messaggio di emergenza sul display OLED:

Sul display compare un messaggio di avviso: "ALERT!"

PULSANTE

Nel progetto è stato implementato un pulsante fisico per simulare il pagamento al termine della sosta.

Il pulsante è cablatto nella configurazione “*pull-down*”, ovvero:

- Un capo è collegato direttamente a GND.
- L'altro capo è collegato a una resistenza da 20Ω in serie e successivamente a un GPIO dell'ESP32 configurato in input digitale.



PULSANTE

FUNZIONAMENTO:

1. All'ingresso del veicolo nel parcheggio, lo stato viene aggiornato a "OCCUPATO" e la sbarra chiusa.
2. Al termine della sosta, l'utente può effettuare il pagamento:
 - tramite **dashboard** → invio messaggio MQTT su topic "pagamento".
 - tramite **pulsante fisico** → rilevamento digitalRead(pin).
3. Avvenuto il pagamento, si attiva il servomotore per l'apertura della barriera.

DASHBOARD

La *Dashboard* è stata progettata per comunicare con il sistema attraverso il protocollo MQTT, utilizzando una connessione Wi-Fi e un broker locale eseguito in un container Docker. L'interfaccia è stata realizzata tramite **Node-RED**, consentendo la gestione remota di più funzionalità del sistema.

Essa è collegata direttamente a:

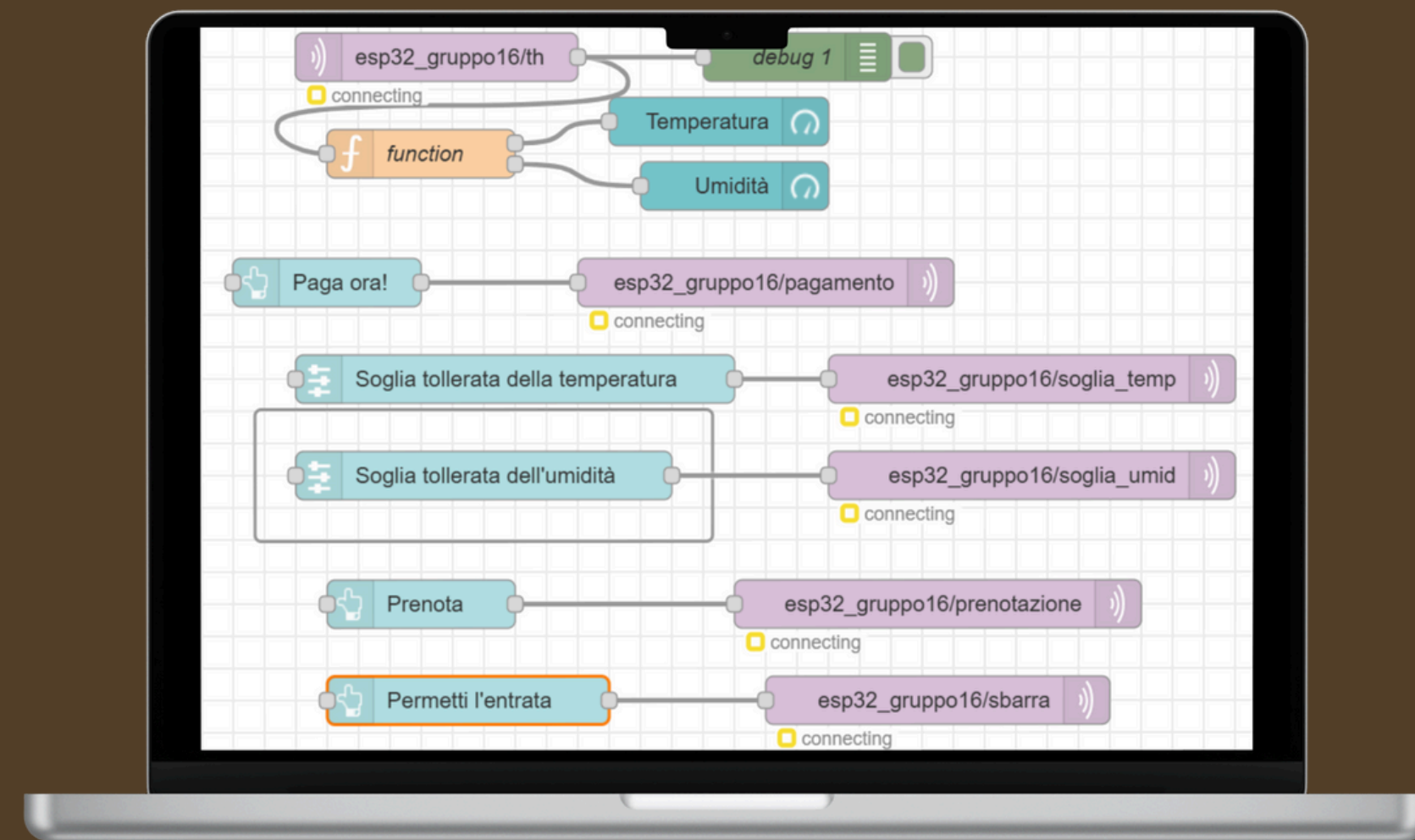
- Il **sensore DHT22** per il monitoraggio in tempo reale di temperatura e umidità.
- Le **soglie di temperatura** e **umidità**, per la simulazione di condizioni meteo estreme
- La **variabile "pagamento"**, che consente di simulare l'avvenuto pagamento per lo stazionamento.
- La **variabile "prenotato"**, che indica se il posto auto è stato riservato da remoto.



DRASHBOARD E FUNZIONALITÀ

L'utente può:

- **Visualizzare** le condizioni ambientali del garage.
- **Modificare** le soglie di temperatura e umidità, attivando automaticamente il sistema di allarme quando vengono superate nell'ambiente circostante.
- **Prenotare** il posto auto da remoto: in questo caso il display mostrerà "PRENOTATO" e il LED passerà a rosso e, successivamente, permettere l'entrata.
- **Pagare** direttamente dalla dashboard, che abilita la riapertura della barriera in uscita.



CODICE

Tutto il codice del progetto è stato strutturato in maniera modulare, adottando un approccio orientato agli oggetti. Dove necessario, sono state create classi dedicate alla gestione dei singoli dispositivi, ciascuna responsabile del controllo e dell'interazione con il relativo componente.

Queste classi sono state implementate in file distinti (.py) e successivamente caricate direttamente sull'ESP32.

Tutte le classi hanno metodi definiti per ogni funzionalità (es. lettura dati, controllo stato, attivazione eventi).

L'importazione delle classi all'interno del file ***main.py*** consente la gestione centralizzata della logica del sistema, mantenendo al contempo un livello di separazione delle responsabilità.

```
display.show_logo()
display.oled.text(str(temperatura) + "C", 10, 10)
display.oled.text(str(umidita) + "%", 10, 30)
display.oled.show()
red.off()
yellow.off()
green.on()
```

```
if (stato.libero and waiting) and not Situazione:
    Situazione = True
    green.off()
    red.off()
    yellow.on()
    angle(90)
    speaker.play_ladyg()
```

```
if (stato.libero and not Situazione):
    Situazione = True
    display.clear()
    display.oled.text(str(temperatura) + "C", 10, 10)
    display.oled.text('OCCUPATO', 30, 30)
    display.oled.show()
    red.off()
    yellow.off()
    green.on()
```

CODICE

Questa struttura modulare ha permesso:

- Un controllo su ciascun dispositivo.
- Un'agevole manutenzione del codice.
- La possibilità di estendere facilmente il sistema in futuro (es. aggiunta nuovi sensori o funzionalità).

L'approccio rispetta i principi fondamentali dell'ingegneria del software in ambito embedded e IoT.



```

# ----- MESSA IN FUNZIONE DEL SISTEMA -----
while True:

    foundMatch = False    # variabile che rende unica la condizione da eseguire nel ciclo

    try:
        if not wifi.check_connection(): # controlliamo che il dispositivo sia ancora connesso
            display.clear()             # in caso contrario, lo riconnettiamo
            display.oled.text('Riconnessione...', 0, 30) # mostriamo 'Riconnessione' a schermo
            display.oled.show()

        mqtt.check_msg() # attraverso MQTT, controlliamo la presenza di messaggi

        ''' Prendiamo i dati dai sensori per elaborare lo stato in cui si trova il garage.
            Dai due sensori ad ultrasuoni leggiamo la distanza misurata,
            da cui elaboriamo la presenza di eventuali veicoli all'ingresso o all'interno della struttura.'''

        distance_outside = sensor_outside.distance_cm() # lettura della distanza misurata dal sensore esterno
        distance_inside = sensor_inside.distance_cm()   # lettura della distanza misurata dal sensore interno
        ht.measure() # lettura dei valori di temperatura e umidità della struttura

        print('Distanza fuori:', distance_outside, 'cm')
        print('Distanza dentro:', distance_inside, 'cm')
        print('Temperatura: ', ht.temperature(), 'C, Umidità: ', ht.humidity(), '%\n')
        temperatura = ht.temperature()
        umidita = ht.humidity()

        # utilizzando MQTT, pubblichiamo i dati riguardanti i valori di temperatura ed umidità registrati

        msg = "Temperatura: {} C, Umidità: {} %".format(ht.temperature(), ht.humidity())
        mqtt.publish(msg)

        ''' Valutiamo lo stato del garage attraverso due variabili booleane.
            Se la distanza misurata dal sensore interno è maggiore di 18 cm, il garage è LIBERO
            Se il sensore esterno misura un valore minore di 10 cm, è stato rilevato un veicolo all'esterno in WAITING'''

        stato.libero = distance_inside > 18 # valutiamo la condizione di LIBERO
        waiting = distance_outside < 10     # valutiamo la condizione di WAITING

        '''Condizioni di emergenza:
            se la temperatura o l'umidità rilevate superano una certa soglia (modificabile in app)
            il sistema va in allarme'''

        allarmeTemperatura = ht.temperature() > stato.soglia_temp # default: 50
        allarmeUmidità = ht.humidity() > stato.soglia_umid # default: 90

```

CODICE - MAIN

```
# -----VALUTAZIONE STATI -----

'''La prima condizione che valutiamo è quella d'emergenza:
se una delle variabili allarmeTemperatura o allarmeUmidità è vera, bisogna evacuare la struttura.
Eseguiamo questo codice in maniera ciclica fino a fine emergenza'''

if (allarmeTemperatura or allarmeUmidità) and not foundMatch:
    foundMatch = True
    display.clear()
    display.oled.text('ALERT!', 40, 30) # mostriamo ALERT sull'OLED
    display.oled.show()

    print('ALLARME. EVACUARE')
    set_angle(90) # alziamo la sbarra per permettere l'evacuazione
    red.on() # facciamo lampeggiare il semaforo
    yellow.on()
    green.on()
    buzzer.play_alarm() # suoniamo l'allarme
    red.off()
    yellow.off()
    green.off()

'''Seconda condizione: il garage è libero ed è stato prenotato'''

if (stato.prenotato and stato.libero) and not foundMatch:
    foundMatch = True
    print("IF DEL PRENOTATO")
    display.clear()
    display.oled.text(str(temperatura) + "C " + str(umidita) + "%", 1, 1)
    display.oled.text('PRENOTATO', 30, 30)
    display.oled.show() # mostriamo l'avvenuta prenotazione sull'OLED ed i valori di temperatura ed umidità
    yellow.off()
    green.off()
    red.on() # accendiamo il semaforo rosso
    set_angle(0) # chiudiamo o manteniamo chiusa la sbarra
```

CODICE- MAIN

```
'''Terza condizione: è stato effettuato il pagamento (il veicolo deve uscire) e l'interno è occupato'''
```

```
if stato.pagamento and not foundMatch:  
    foundMatch = True
```

```
    if not stato.libero:                # rivalutiamo l'if finchè il veicolo non è fuori dalla struttura  
        display.clear()  
        display.oled.text('PAGATO',40,30) # mostriamo la riuscita del pagamento  
        display.oled.show()  
        red.off()  
        yellow.on()  
        set_angle(90)                  # apriamo la sbarra per permettere l'uscita  
        buzzer.play_ladyg()            # suoniamo ABRACADABRA sul buzzer  
    if stato.libero:  
        stato.pagamento = False
```

```
'''Quarta condizione: il garage è libero e nessun veicolo è posto all'esterno'''
```

```
if (stato.libero and not waiting) and not foundMatch:  
    foundMatch = True  
    set_angle(0)                        # chiudiamo o manteniamo chiusa la sbarra  
    display.show_logo()  
    display.oled.text(str(temperatura) + "C", 1, 1, 0)  
    display.oled.text(str(umidita) + "%", 1, 10, 0)  
    display.oled.show()                 # mostriamo la temperatura, l'umidità ed il nostro logo  
    red.off()  
    yellow.off()  
    green.on()                          # accendiamo il semaforo verde
```

CODICE- MAIN

```
'''Quinta condizione: il garage è libero ed un veicolo sta aspettando all'esterno'''

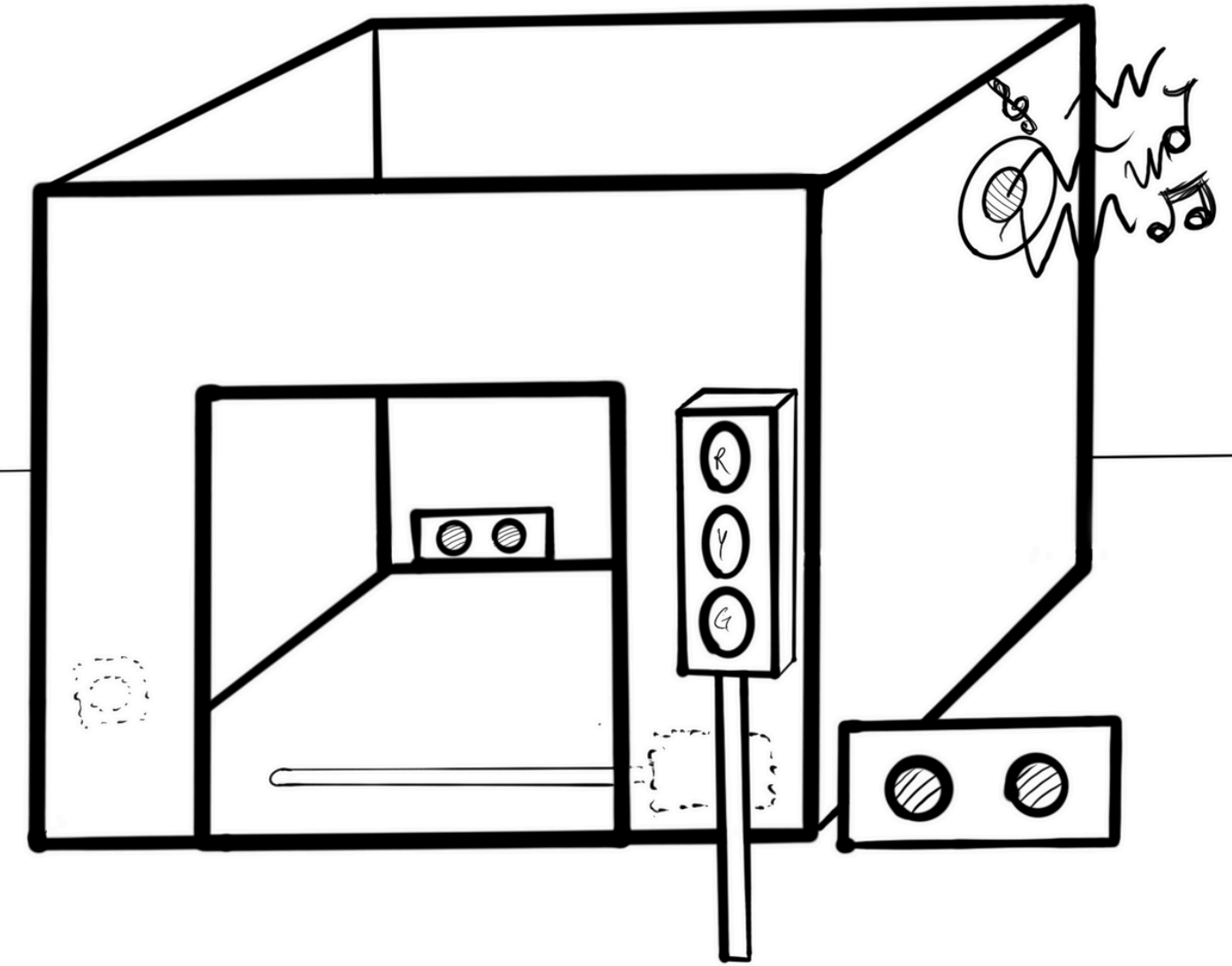
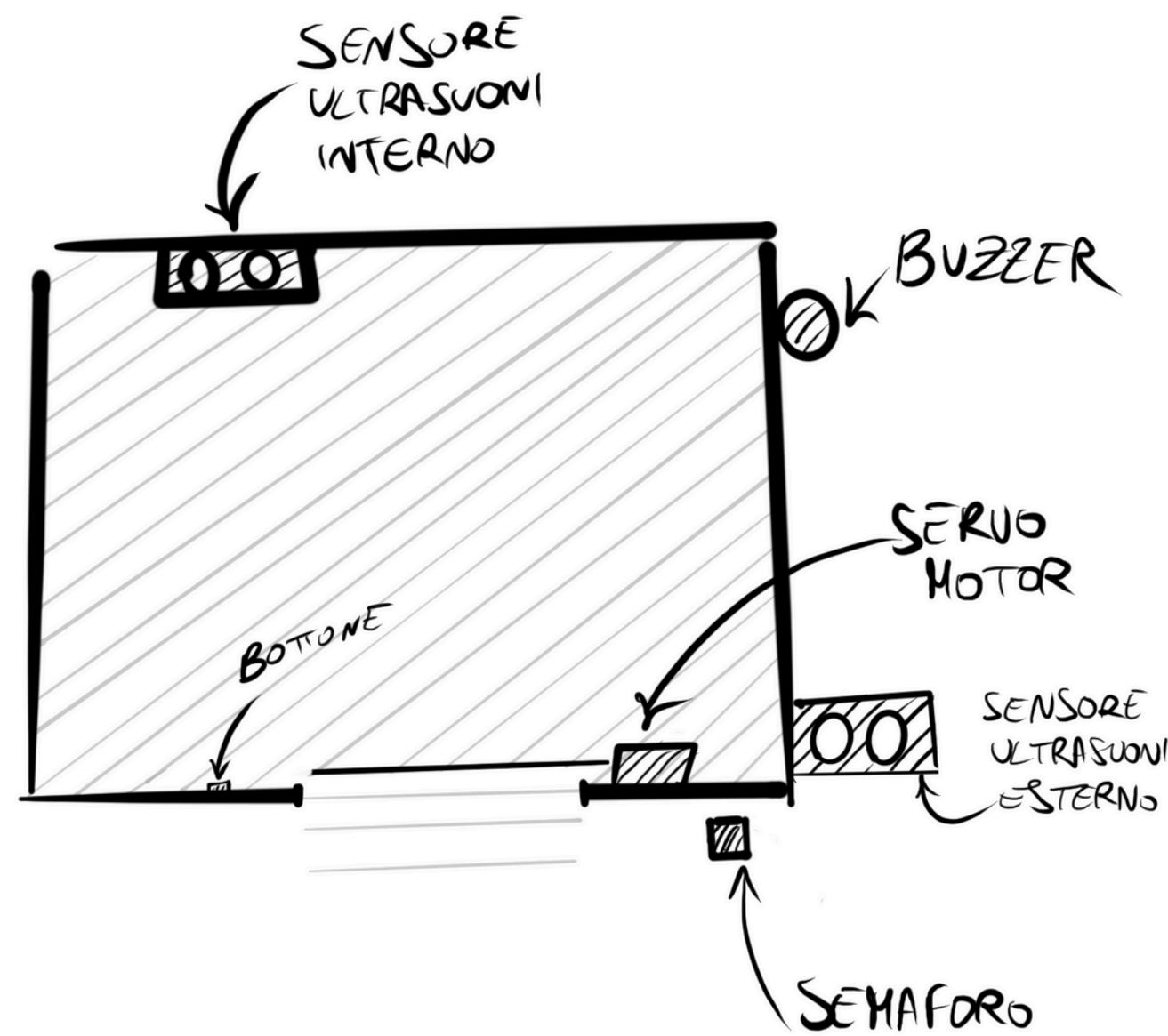
    if (stato.libero and waiting) and not foundMatch:
        foundMatch = True
        green.off()
        red.off()
        yellow.on()          # accendiamo il semaforo giallo
        set_angle(90)        # apriamo la sbarra
        buzzer.play_ladyg()   # suoniamo ABRACADABRA sul buzzer

'''Sesta condizione: il garage è occupato'''

if not stato.libero and not foundMatch:
    foundMatch = True
    display.clear()
    display.oled.text(str(temperatura) + "C " + str(umidita) + "%",1,1)
    display.oled.text('OCCUPATO',30,30)
    display.oled.show()      # mostriamo OCCUPATO, temperatura ed umidità sull'OLED
    yellow.off()
    green.off()
    red.on()                 # accendiamo il semaforo rosso
    set_angle(0)            # chiudiamo o manteniamo chiusa la sbarra

except OSError as ex:
    print('Errore:', ex)
sleep(0.5)
```

CODICE- MAIN



Modellino

Il modellino tridimensionale del garage è stato realizzato con un design caratterizzato da pareti di colore avorio e un pavimento dall'effetto pietra. L'intero prototipo è collocato su una base rialzata, concepita per ospitare la breadboard connessa ai sensori necessari al funzionamento del sistema. Nel suo insieme, il modellino offre una percezione spaziale e sequenziale delle dinamiche operative del progetto, agevolando la comprensione del flusso logico e funzionale del prototipo.